

Auteur : Briek Wozniak

Inhoud van de colleges

De eerste les begon met een inleiding op hoe kritieke infrastructuur aangevallen kan worden. Naast fysieke opties zoals het uitschakelen van essentiële installaties of het ontvoeren van personeel bespraken we vooral digitale methoden. En hoewel veel netwerken van kritieke systemen (kerncentrales, wapenfabrikanten, energieproducenten, ...) afgeschermd en niet aan het internet gekoppeld zijn, bleken er toch praktische manieren te zijn om malware binnen te brengen. Een voorbeeld is het binnensmokkelen van een USB-stick. Dit betekent dus dat het zeker mogelijk is om malware binnen te smokkelen bij kritieke infrastructuur.

Dit leidt dan naar het hoofdonderwerp van de les: kwetsbaarheden in laag-niveau talen zoals C en assembly, die nog heel vaak in kritieke systemen worden gebruikt. Om te begrijpen hoe die misbruikt kunnen worden, hebben we eerst bekeken hoe een x86-processor werkt: instructies worden uit het geheugen gelezen, verwerkt en het resultaat in registers weggeschreven. Dit herhaalt zich voortdurend. In het begin was programmeren heel moeilijk dus werden er talen zoals assembly en C geïntroduceerd (met C die naar assembly wordt vertaald). Deze talen worden dan rechtstreeks omgezet naar machine code om zo development gemakkelijker te maken.

Een belangrijk zwak punt is dat C geen automatische controles op onder andere buffer-overflows uitvoert. Een overflow ontstaat wanneer je meer data naar een reservering op de stack schrijft dan daarvoor bedoeld is, daardoor kun je aangrenzend geheugen overschrijven. Omdat functies in C met returnadressen werken, kan een overschreven returnadres ertoe leiden dat de uitvoering na het terugkeren naar een door een aanvaller gekozen locatie springt. Hierdoor kunnen beveiligingschecks overgeslagen worden of zelf eigen code uitgevoerd worden. In grote codebases zijn zulke fouten ook nog eens lastig te vinden.

We bespraken enkele mogelijke oplossingen: expliciet het originele returnadres bewaren voordat input wordt verwerkt; veiliger stringfuncties gebruiken (zoals `strncpy` die lengte controleert en strings correct afsluit met `"\0"`); en stack-guards (canaries) die veranderingen aan een verborgen waarde detecteren (wanneer ze zelf overschreven worden) en bij manipulatie het programma laten stoppen. Canaries zijn echter niet waterdicht: als een aanvaller de waarde kent, kan die worden hersteld in de overflow en wordt de fout dus niet gedetecteerd.

Andere tegenmaatregelen zijn ASLR (Address Space Layout Randomization) waarbij adresruimtes worden gerandomiseerd zodat aanvallers niets kunnen hardcoden en privilege rings, die code in trustniveaus scheiden (bijv. hypervisor, system management mode). Beide benaderingen hebben hun beperkingen: randomness kan voorspelbaar worden, en fouten in hooggeprivilegieerde firmware zijn erg kritisch. Hardwareoplossingen (bijv. cryptografische modules op het moederbord) bieden extra bescherming.

Het is dus belangrijker om te kiezen voor memory-safe talen. Rust lost veel van deze zwaktes op: de compiler voert bounds-checks uit voor lezen en schrijven en voorkomt onder andere use-after-free. Conclusie van de les: geheugenonveilige talen veroorzaken veel problemen waarvoor oplossingen bestaan, maar het gebruik van memory-safe talen zoals Rust voorkomt veel van deze kwetsbaarheden al bij de oorzaak.

De tweede les sloot aan bij de eerste en ging dieper in op het veiligstellen van memory, met de nadruk op confidential computing. Dit beschermt de data van een applicatie door deze in een geïsoleerde omgeving (*enclave*) te plaatsen. Enkel de eigenaar van de enclave kan de data daarin lezen; zelfs de kernel en de hypervisor hebben geen toegang. Een voorbeeld hiervan is de Intel SGX-enclave.

Het grote voordeel van deze enclaves is dat men enkel hoeft te vertrouwen op de eigen code en op de implementatie van de enclave door de fabrikant, aangezien enkel het programma zelf toegang heeft tot de data binnen de enclave. Dit principe noemen we encapsulatie.

Een ander belangrijk onderdeel binnen dit concept is *attestatie*. Hierbij wordt met cryptografische sleutels gecontroleerd dat de code binnen de enclave niet is aangepast en dat de data vertrouwelijk blijft. Vervolgens bespraken we Sancus, een architectuur die confidential computing op hardwareniveau afdwingt. Voor de isolatie wordt gebruikgemaakt van hardware-enforced access control, waarbij externe code enkel via een entry point toegang krijgt. De module bestaat dus uit een entry point waarmee externe code contact kan leggen met de beschermde code, een text section en uiteraard de belangrijkste: de protected section.

Voor encryptie wordt RSA (public/private key) gebruikt voor veilige communicatie. Er is ook een kill switch waarmee alle data in de protected section kan worden gewist. Dit is nuttig wanneer er iets misloopt, zodat een extern programma de sectie alsnog kan leegmaken. Voor attestatie worden opnieuw cryptografische sleutels gebruikt, evenals een *Key Derivation Function* (KDF), die een nieuwe sleutel kan genereren uit een bestaande met extra variabelen.

Er bestaan verschillende platformen voor confidential computing, zoals Intel SGX, AWS Nitro Enclaves en AMD SEV-SNP. Voor softwareontwikkeling binnen deze enclaves is er het *Enclave Development Platform* (EDP). Dit kan volledig in Rust worden geprogrammeerd en gebruikt de *Enclave Runner*, die de enclave start, beheert en vernietigt. De communicatie met de enclaves verloopt via system calls. Daarnaast bestaat er een besturingssysteem, *Enclave OS*, dat automatisch alles veilig binnen enclaves plaatst. Fortanix voorziet al deze tools (Enclave OS, EDP en cloud-integraties voor Azure, AWS en GCP).

De laatste les ging over de zwakke punten van Intel SGX, want dit systeem lost zeker niet alle beveiligingsproblemen op. Een van de grootste tekortkomingen is dat de Trusted Computing Base (TCB) te groot is. Hoe meer code en systeemcomponenten je moet vertrouwen, hoe groter de kans op fouten en kwetsbaarheden. Veel klassieke problemen, zoals out-of-bounds reads en writes, blijven mogelijk als je geen memory-safe taal gebruikt. De beste aanpak voor meer veiligheid is daarom een combinatie van een memory-safe taal (zoals Rust) en een verkleinde, betrouwbare TCB binnen een confidential computing-omgeving (zoals SGX met attestation).

We bespraken ook de architectuur van een enclave, maar dat laat ik hier buiten beschouwing om het kort te houden. Belangrijker zijn de mogelijke aanvalsscenario's. Als de kernel of hypervisor wordt gecompromiseerd, vormt dat uiteraard een ernstig risico. Verder kunnen enclaves kwetsbaar zijn tijdens shutdowns of resets, wanneer de enclave-state kan worden gemanipuleerd, een probleem dat bijzonder moeilijk op te lossen is.

Daarnaast bestaan er controlled-channel attacks (zoals page-fault leakage): wanneer de enclave geheugenpagina's opvraagt die niet geladen zijn, kan de kernel zien welke pagina's dat zijn en zo indirect informatie afleiden over de werking van de enclave. Ook CPU-gebonden aanvallen, zoals het uitlezen van de cache, kunnen gevoelige informatie blootleggen.

De conclusie van deze lessen is duidelijk: SGX biedt waardevolle bescherming, maar is geen silver bullet. Gebruik Rust of een andere memory-safe taal om klassieke fouten te vermijden, beperk de TCB zoveel mogelijk, en ontwerp software zonder afhankelijkheid van geheime waarden in control flow. Sommige low-level aanvallen (zoals controlled channels en speculative leaks) zullen altijd moeilijk volledig te voorkomen zijn, dus goed systeemontwerp en aanvullende mitigaties blijven noodzakelijk. Het belangrijkste principe blijft: do not branch on secrets, voer nooit verschillende codepaden uit op basis van geheime data.

Belangrijkste lessons learned

1. Wat ik persoonlijk heel interessant vond, was de uitleg over de kerncentrale en hoe het binnensmokkelen van een simpele USB-stick, gevolgd door het gebruiken van een aantal exploits, ervoor zorgde dat de hele kerncentrale platlag. Hieruit bleek dat je technisch gezien veel moeite kunt steken in de digitale beveiliging van systemen, maar dat fysieke toegang nog steeds een groot gevaar vormt en dat je daar ook fysieke beveiliging voor moet voorzien.
2. Ik wist al uit andere vakken dat talen zoals C en assembly potentieel problemen kunnen veroorzaken, zoals memory leaks, out-of-bounds reads/writes en use-after-free. Daarom hebben we bij GOGP al eens gewerkt met een memory-safe taal (Rust). Deze talen voorkomen zulke problemen doordat de compiler strikte checks uitvoert op geheugentoegang en data-eigendom. Dit principe kende ik dus al oppervlakkig uit de lessen, maar deze gastcolleges toonden concreet aan hoe groot de impact van zulke fouten kan zijn, bijvoorbeeld bij aanvallen zoals Stuxnet en vergelijkbare voorbeelden.
3. Wat ik ook heb geleerd, is dat vertrouwensmechanismen zoals Intel SGX geen "silver bullet" zijn. Hoewel enclaves bescherming bieden tegen veel aanvallen, blijft de Trusted Computing Base (TCB) vaak te groot, waardoor fouten in onderliggende componenten alsnog tot exploits kunnen leiden. Dit was grotendeels nieuw voor mij, ik wist wel dat hardware-ondersteunde beveiliging bestond, maar van enclaves had ik nog nooit gehoord. Het viel me op dat het toevoegen van enclaves als beveiligingsmaatregel ook weer een nieuw platform voor exploits kan bieden, je probeert een probleem op te lossen en creëert daarmee mogelijk nieuwe problemen.
4. Een andere belangrijke les was dat beveiliging altijd gelaagd moet zijn. Geen enkele oplossing is volledig waterdicht, maar door meerdere mechanismen te combineren zoals stack canaries, ASLR, privilege rings en cryptografische hardwaremodules, kan het risico sterk worden beperkt. Veel van deze technieken kende ik nog niet. Het was interessant om te zien hoe ze in de praktijk samen worden ingezet in kritieke infrastructuren en hoe "eenvoudig" sommige systemen soms zijn. Neem bijvoorbeeld de canaries, dit systeem is eigenlijk heel eenvoudig en voor de hand liggend, maar toch zeer effectief. Ik vond het daarom leuk dat zo'n simpel systeem dat letterlijk een random waarde neerschrijft en controleert toch een extra laag beveiliging toevoegt. Dit principe zie je ook terug in real-world besturingssystemen zoals Linux, waar address randomization en gebruikersrechten samen worden gebruikt om exploits te bemoeilijken.

Zelfreflectie

Een concreet voorbeeld van een project waar ik momenteel aan werk, is een game die ik samen met een paar vrienden aan het ontwikkelen ben. Dit doen we in de open-source game-engine Godot, met C# als programmeertaal. Het onderdeel uit mijn code dat ik graag wil bespreken, is mijn save/load-systeem. Wanneer de speler zijn virtuele wereld afsluit, wordt alle vooruitgang opgeslagen in JSON-formaat in een JSON-bestand.

Technisch gezien werkt dit systeem perfect, alle data wordt gestructureerd weggeschreven en later opnieuw correct ingeladen. Toch ben ik, na de lessen over beveiliging, tot het besef gekomen dat deze aanpak niet echt veilig is. Op het moment dat ik dit systeem implementeerde was ik vooral gefocust op de functionaliteit en minder op de veiligheid. Zoals ik al zei, werk ik met C#, wat op zich een memory-safe taal is. Daardoor ontstaan er niet snel problemen zoals out-of-bounds errors. Maar aangezien de save-file wordt opgeslagen als een leesbaar JSON-bestand, is deze wel vatbaar voor externe manipulatie. Dit probleem hebben we ook in de lessen besproken.

Mijn save-systeem vertrouwt namelijk volledig op de integriteit van het JSON-bestand. Er is geen enkele vorm van authenticatie, encryptie, of integriteitscontrole om te verifiëren of de data tussentijds is aangepast. Hierdoor kan een gebruiker eenvoudig het bestand openen in een teksteditor en bijvoorbeeld de hoeveelheid in-game valuta of de voortgang aanpassen. In principe vormt dit geen direct risico voor het systeem zelf buiten dat er een beetje vals gespeeld kan worden, maar het toont wel aan dat er een afwezigheid is van vertrouwensmechanismen.

Tijdens de lessen leerden we dat vertrouwensmodellen, zoals Intel SGX, niet perfect zijn, maar wel een extra laag bescherming kunnen bieden tegen manipulatie. In mijn geval zou een eenvoudiger alternatief, zoals het toevoegen van een cryptografische handtekening of een hash van de opgeslagen data, al een aanzienlijke verbetering betekenen.

Een andere belangrijke les die ik wil toepassen, is het belang van gelaagde beveiliging. Mijn oorspronkelijke implementatie vertrouwde volledig op de applicatielogica, er was geen enkele controle buiten de code zelf. In een verbeterde versie zou ik meerdere beveiligingslagen toevoegen. Allereerst zou ik de JSON-data versleutelen met een symmetrisch encryptieschema (zoals AES), zodat de inhoud niet zomaar leesbaar of manipuleerbaar is. Daarnaast zou ik bij het laden van de data een hashcontrole uitvoeren om te verifiëren dat het bestand niet gewijzigd is. Zelfs als iemand fysieke toegang heeft tot het bestand, wordt het dan veel moeilijker om de data te manipuleren zonder dat het spel dit detecteert.

Tot slot hebben deze lessen mij doen inzien dat "eenvoudige" beveiligingsmaatregelen vaak verrassend effectief kunnen zijn. Net zoals stack canaries in besturingssystemen die buffer overflows kunnen detecteren, kan in mijn project een relatief simpele controle (bijvoorbeeld het toevoegen van een checksum op het einde van de save file) al een groot verschil maken.

Het belangrijkste inzicht dat ik hieruit meeneem, is dat je nooit een perfect beveiligd systeem kunt bouwen. Encryptie en andere beveiligingstechnieken kunnen na verloop van tijd door nieuwe technologieën worden gekraakt. Toch kan het toevoegen van meerdere verdedigingslagen vaak al voldoende zijn om veelvoorkomende exploits te voorkomen.

Als ik dit save-systeem opnieuw zou implementeren (wat waarschijnlijk nodig zal zijn), zou ik zeker encryptie gebruiken om het bestand zelf te versleutelen, zodat een gebruiker niet zomaar zijn savefile kan bewerken. Daarnaast zou ik een checksum of hash toevoegen om te controleren of het bestand

niet is aangepast. Deze lessen hebben mij geleerd dat zelfs in kleine projecten beveiliging nooit een bijzaak mag zijn, maar een essentieel onderdeel dat vanaf het begin van de ontwikkeling in overweging moet worden genomen.

Appreciatie van de lessenreeks

Ik vond de eerste les bijzonder interessant, vooral omdat we toen een boeiende casus over Stuxnet bespraken. We gingen op een interactieve manier op zoek naar mogelijke manieren waarop een stukje code misbruikt kon worden. Dat maakte het onderwerp meteen tastbaar. Hoewel ik erg hou van interactieve lessen, vond ik wel dat de gastprofessor soms wat te lang wachtte op meerdere antwoorden of oplossingen uit de groep, waardoor het af en toe lastig was om de aandacht erbij te houden.

In de latere lessen vond ik het jammer dat we minder met concrete voorbeelden werkten en dat de focus meer verschoof naar abstracte concepten, vooral rond enclaves. Ik begrijp dat de gastprofessor deels ook de bedoeling had om Fortranix en de technologieën waar hij daar mee werkt wat te belichten. Toch vond ik het teleurstellend dat we op dat onderdeel bleven hangen en minder andere mogelijke aanvalsmethoden of beveiligingsstrategieën hebben behandeld. Dit vond ik vooral jammer omdat ik een sterke interesse heb in cybersecurity en deze richting als master wil volgen. Tot nu toe wordt dit onderwerp in onze opleiding slechts oppervlakkig behandeld en wordt er nergens echt in veel detail gegaan, dus ik had graag wat meer variatie gezien in de besproken thema's.

Wat ik echter wel waardeerde, was dat de gastprofessor af en toe interessante randinformatie deelde. Zo ging hij bijvoorbeeld dieper in op verschillende vormen van encryptie, nadat iemand uit de groep aangaf dat we daar nog niet veel over hadden geleerd. Het was duidelijk dat hij enorm veel kennis had over het onderwerp, en telkens wanneer iemand een vraag stelde, volgde er een heldere en uitgebreide uitleg.

Ik had zelf nog nooit gehoord van het concept Confidential Computing, dus het was bijzonder leerzaam om hier meer over te ontdekken. Soms ging de uitleg misschien wat diep in op de technische details, maar ik vermoed dat er zeker studenten waren die dat juist wisten te appreciëren.

Als conclusie kan ik zeggen dat ik de lessen zeer interessant vond. Hoewel het enigszins jammer was dat we slechts één specifiek aspect zo uitgebreid behandelden, begrijp ik die keuze wel. Het onderwerp was immers niet zo simpel. Al bij al heb ik veel bijgeleerd, zowel op theoretisch als praktisch vlak, en heeft het mijn interesse in cybersecurity alleen maar verder uitgebreid.